

使用 Dataproc 和 Google Managed Service for Apache Kafka 即時處理 IoT 資料

來源：<https://codelabs.developers.google.com/codelabs/iot-data-processing-using-dataproc-and-managed-kafka?hl=zh-tw>

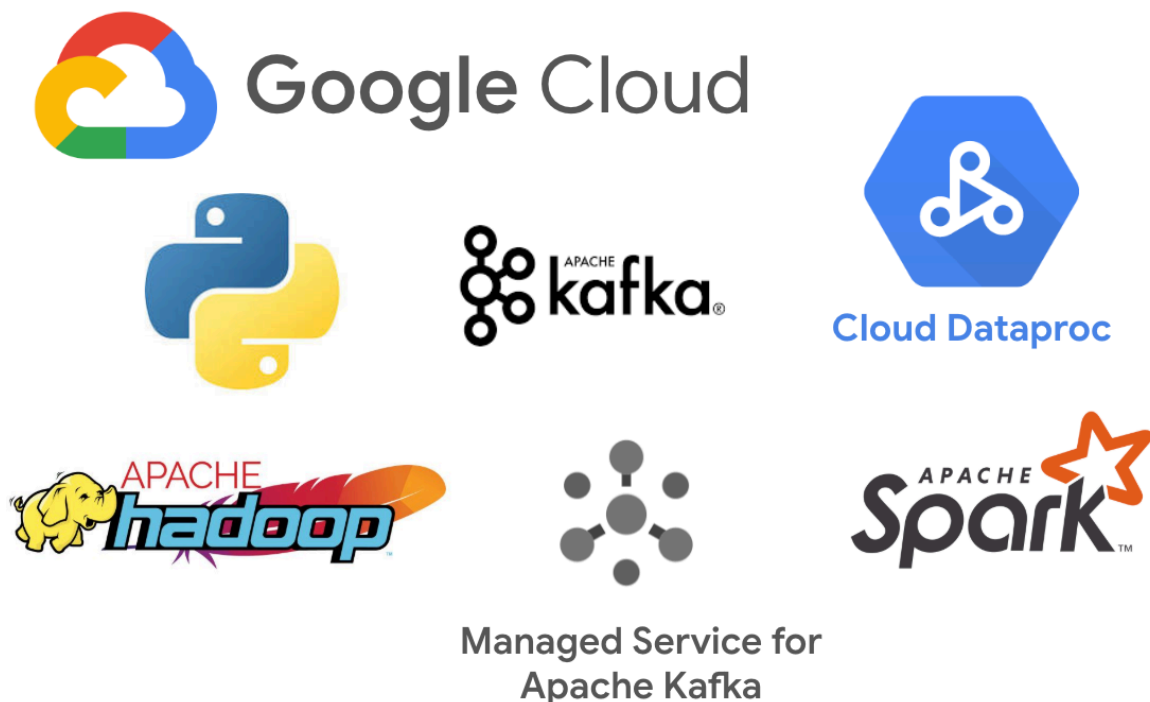
步驟數：9

在本程式碼研究室中，您將使用 Managed Kafka 和 Dataproc，從即時物聯網事件產生洞察資料。

目錄

1. [簡介](#)
2. [總覽](#)
3. [設定 Google 代管 Kafka](#)
4. [設定發布者](#)
5. [發布至代管 Kafka](#)
6. [設定 Dataproc 叢集](#)
7. [使用 Dataproc 處理 Kafka 訊息](#)
8. [清除所用資源](#)
9. [恭喜](#)

1. 簡介



上次更新時間：2024 年 6 月 10 日

背景

從智慧住宅解決方案到工業感應器，物聯網 (IoT) 裝置會在網路邊緣產生大量資料。這類資料非常適合用於各種用途，例如裝置監控、追蹤、診斷、監控、個人化、車隊最佳化等。Google Managed Service for Apache Kafka 提供可擴充且持久的解決方案，以相容於 OSS 的方式輕鬆安全地擷取及儲存這類持續串流的資料，而 Google Cloud Dataproc 則可使用 Apache Spark 和 Hadoop 叢集處理這些大型資料集，進行資料分析。

建構項目

在本程式碼實驗室中，您將使用 Google Managed Service for Apache Kafka、Dataproc、Python 和 Apache Spark 建構 IoT 資料處理管道，進行即時分析。管道會執行下列動作：

- 使用 GCE VM 將 IoT 裝置的資料發布至 Managed Kafka 叢集
- 將代管 Kafka 叢集的資料串流至 Dataproc 叢集
- 使用 Dataproc Spark Streaming 工作處理資料

課程內容

- 如何建立 Google 代管 Kafka 和 Dataproc 叢集
- 如何使用 Dataproc 執行串流工作

軟硬體需求

- 已設定專案的有效 GCP 帳戶。如果沒有帳戶，可以申請[免費試用](#)。
- 已安裝並設定 gcloud CLI。您可以按照[在作業系統上安裝 gcloud CLI](#)的操作說明進行。

- 在 GCP 專案中啟用 [Google Managed Kafka](#) 和 [Dataproc](#) 的 API。
-

2. 總覽

在本程式碼研究室中，我們將以虛構公司 DemoIoT Solutions 的故事為例。DemoIoT Solutions 提供感應器裝置，可測量及傳輸溫度、濕度、壓力、光照強度和位置資料。他們希望設定管道來處理這項資料，以便向顧客顯示即時統計資料。他們可以運用這類管道，為客戶提供各種服務，例如監控、自動建議、快訊，以及客戶安裝感應器地點的深入分析。

為此，我們將使用 GCE VM 模擬 IoT 裝置。裝置會將資料發布至 Google Managed Kafka 叢集中的 Kafka 主題，而 Dataproc 串流工作會讀取並處理這些資料。「先決條件設定」和後續頁面會引導您完成所有這些步驟。

前置設定

1. 找出專案的名稱和編號。如需參考資料，請參閱「[找出專案名稱、編號和 ID](#)」。
2. 虛擬私有雲子網路。這樣一來，GCE VM、Kafka 叢集和 Dataproc 叢集之間就能連線。請按照[這個](#)頁面的說明，使用 gcloud CLI 列出現有子網路。如有需要，請按照[建立自動模式虛擬私有雲網路](#)的步驟操作，在每個 Google Cloud 區域中建立具有子網路的虛擬私有雲網路。不過，在本程式碼研究室中，我們只會使用單一區域的子網路。
 - 在這個子網路中，請確保有防火牆規則允許來自 tcp:22 的所有輸入流量，這是 SSH 的必要條件。建立網路時，您可以在「防火牆規則」部分選取這項規則，請務必選取。
3. GCS bucket。您需要存取 Google Cloud Storage bucket，才能儲存 Dataproc 工作資源並保存已處理的資料。如果沒有，您可以在 GCP 專案中[建立](#)一個。

填入環境變數

在執行 gcloud CLI 的終端機中，填入這些環境變數，以便稍後使用。

```
export PROJECT_ID=<project-id>
export PROJECT_NUMBER=<project-number>
export REGION=<region>
export ZONE=<zone>
export SUBNET_PATH=<subnet-path>
export BUCKET=gs://<bucket-name>
```

更改下列內容：

- `<project-id>` 替換為您設定的 GCP 專案名稱。
- `<project-number>` 替換為必要條件步驟 1 中的專案編號名稱。
- `<region>` 改成您要使用的[可用區域和可用區](#)名稱。舉例來說，我們可以使用 `us-central1`。
- `<zone>`，其中 [Available regions and zones](#) 是您先前選取區域下方的區域名稱。舉例來說，如果您選取 `us-central1` 做為區域，則可以使用 `us-central1-f` 做為可用區。這個可用區將用於建立模擬 IoT 裝置的 GCE VM。確認[可用區位於您選擇的地區](#)。
- `<subnet-path>`，並提供必要條件步驟 2 中的子網路完整路徑。這個值的格式必須為 `projects/<project-id>/regions/<region>/subnetworks/<subnet-name>`。
- 將 `<bucket-name>` 改成先決條件步驟 3 中的 GCS bucket 名稱。

注意：後續章節可能會直接使用這些變數的值，因此請確保這些變數已在您執行指令的終端機中填入。

3. 設定 Google 代管 Kafka

本節將設定 Google Managed Kafka 叢集，部署 Kafka 伺服器，並在這個伺服器上建立主題，以便發布及讀取 IoT 資料 (訂閱後)。DemoIoT Solutions 可以設定這個叢集，讓所有裝置將資料發布至該叢集。

建立 Kafka 代管叢集

- 建立 Managed Kafka 叢集。這裡的叢集名稱是 `kafka-iot`。

```
gcloud managed-kafka clusters create kafka-iot \  
  --project=$PROJECT_ID \  
  --location=$REGION \  
  --cpu=3 \  
  --memory=12GiB \  
  --subnets=$SUBNET_PATH \  
  --auto-rebalance
```

您應該會收到類似以下的回覆：

```
Create request issued for: [kafka-iot]  
Waiting for operation [projects/<project-id>/locations/<region>/operations/<operation-id>] to  
complete...done.  
Created cluster [kafka-iot].
```

叢集建立作業需要 20 到 30 分鐘。等待這項作業完成。

建立主題

- 在叢集上建立 Managed Kafka 主題。這裡的主題名稱為 `kafka-iot-topic`。

```
gcloud managed-kafka topics create kafka-iot-topic \  
  --cluster=kafka-iot \  
  --location=$REGION \  
  --partitions=10 \  
  --replication-factor=3
```

輸出結果應該會類似下列內容：

```
Created topic [kafka-iot-topic].
```

4. 設定發布者

如要發布至 Managed Kafka 叢集，請設定可存取虛擬私有雲的 Google Compute Engine VM 執行個體，該虛擬私有雲包含 Managed Kafka 叢集使用的子網路。這個 VM 會模擬 DemoIoT Solutions 提供的感應器裝置。

步驟

1. 建立 Google Compute Engine VM 執行個體。這裡的 GCE VM 名稱為 `publisher-instance`。

```
gcloud compute instances create publisher-instance \
  --scopes=https://www.googleapis.com/auth/cloud-platform \
  --subnet=$SUBNET_PATH \
  --zone=$ZONE
```

2. 授予 Google Compute Engine 預設服務帳戶使用 Managed Service for Apache Kafka 的權限。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
  --role=roles/managedkafka.client
```

3. 使用 SSH 連線至 VM。或者，您也可以[使用 Google Cloud 控制台透過 SSH 連線](#)。

```
gcloud compute ssh publisher-instance --project=$PROJECT_ID --zone=$ZONE
```

4. 安裝 Java 來執行 Kafka 指令列工具，並使用這些指令下載 Kafka 二進位檔。

```
sudo apt-get install default-jre wget
wget -O kafka_2.13-3.7.2.tgz https://downloads.apache.org/kafka/3.7.2/kafka_2.13-3.7.2.tgz
tar xzf kafka_2.13-3.7.2.tgz
export KAFKA_HOME=$(pwd)/kafka_2.13-3.7.2
export PATH=$PATH:$KAFKA_HOME/bin
```

5. 下載代管 Kafka 驗證程式庫及其依附元件，並設定 Kafka 用戶端屬性。

```
wget https://github.com/googleapis/managedkafka/releases/download/v1.0.5/release-and-dependencies.zip
sudo apt-get install unzip
unzip -n release-and-dependencies.zip -d $KAFKA_HOME/libs/
find "$KAFKA_HOME/libs/release-and-dependencies" -type f -name "*.jar" -exec cp -n {} "$KAFKA_HOME/libs/" \;
```

```
cat <<EOF> client.properties
security.protocol=SASL_SSL
sasl.mechanism=OAUTHBEARER
sasl.login.callback.handler.class=com.google.cloud.hosted.kafka.auth.GcpLoginCallbackHandler
sasl.jaas.config=org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginModule
required;
EOF
```

如要進一步瞭解發布者機器的設定，請參閱「[設定用戶端機器](#)」。

5. 發布至代管 Kafka

發布者設定完成後，我們就可以在發布者上使用 Kafka 指令列，將一些虛擬資料從 GCE VM (由 DemolIoT Solutions 模擬 IoT 裝置) 發布至 Managed Kafka 叢集。

注意：本節中的指令應從上一節建立的 GCE VM 執行個體執行。如果 SSH 連線中斷，請再次連線，並使用上一節的指令填入 `KAFKA_HOME` 和 `PATH` 環境變數，以便使用 Kafka CLI。

1. 由於我們已透過 SSH 連線至 GCE VM 執行個體，因此需要重新填入 `PROJECT_ID` 變數：

```
export PROJECT_ID=<project-id>
export REGION=<region>
```

更改下列內容：

- `<project-id>` 替換為您設定的 GCP 專案名稱。
 - `<region>`，其中 Kafka 叢集是在該區域建立
2. 使用 `managed-kafka clusters describe` 指令取得 Kafka 啟動伺服器的 IP 位址。這個位址可用於連線至 Kafka 叢集。

```
export BOOTSTRAP=$(gcloud managed-kafka clusters describe kafka-iot --project=$PROJECT_ID --
location=$REGION --format='value(bootstrapAddress)')
```

注意：您可以使用 `echo $BOOTSTRAP` 顯示啟動伺服器位址。格式應為 `bootstrap.kafka-iot.<region>.managedkafka.<project-id>:9092`。舊版 gcloud 可能沒有 `:9092` 後置字元。如果缺少 `:9092` 後置字串，請透過 `export BOOTSTRAP=$BOOTSTRAP:9092` 新增。

3. 列出叢集中的主題：

```
kafka-topics.sh --list \
--bootstrap-server $BOOTSTRAP \
--command-config client.properties
```

您應該會看到下列輸出內容，其中包含先前建立的主題 `kafka-iot-topic`。

```
__remote_log_metadata
kafka-iot-topic
```

4. 複製這段指令碼並貼到新檔案 `publish_iot_data.sh` 中。如要在 GCE VM 上建立新檔案，可以使用 `vim` 或 `nano` 等工具。

```
vi publish_iot_data.sh  
# OR (use any one command)  
nano publish_iot_data.sh
```

```
#!/bin/bash

NUM_MESSAGES_PER_DEVICE=20
NUM_IDS=10
start_time=$(date -d "2025-03-01T00:00:00Z" +%s)
message_interval=10

generate_sensor_data() {
    local temperature=$(printf "%.1f" $(echo "scale=1; 20 + $RANDOM % 100 / 10.0" | bc -l))
    local humidity=$(printf "%.1f" $(echo "scale=1; 50 + $RANDOM % 500 / 10.0" | bc -l))
    local pressure=$(printf "%.1f" $(echo "scale=1; 1000 + $RANDOM % 500 / 10.0" | bc -l))
    local light_level=$((RANDOM % 1000))

    echo "\"temperature\": $temperature,"
    echo "\"humidity\": $humidity,"
    echo "\"pressure\": $pressure,"
    echo "\"light_level\": $light_level"
}

generate_location_data() {
    local latitude=$(printf "%.4f" $(echo "scale=2; 33.0 + $RANDOM % 66" | bc -l))
    local longitude=$(printf "%.4f" $(echo "scale=2; -120.0 + $RANDOM % 66" | bc -l))

    echo "\"latitude\": $latitude,"
    echo "\"longitude\": $longitude"
}

generate_device_status() {
    local battery_level=$((RANDOM % 101))
    local signal_strength=$((RANDOM % 80 - 100))
    local connection_types=("Wi-Fi" "Cellular" "LoRaWAN")
    local connection_type="${connection_types[$((RANDOM % ${#connection_types[@]}))]}"

    echo "\"battery_level\": $battery_level,"
    echo "\"signal_strength\": $signal_strength,"
    echo "\"connection_type\": \"$connection_type\""
}

publish_to_kafka() {
    local device_index=$1
    local message_index=$2
    local device_id="sensor-$(device_index % NUM_IDS)"
    local timestamp=$((start_time + (message_index * message_interval))
    local date=$(date -u -d "@$timestamp" +%Y-%m-%dT%H:%M:%SZ)

    local json_data=$(cat <<EOF
{
    "device_id": "$device_id",
    "timestamp": "$date",
    "location": {
$(generate_location_data)
    },

```

```

    "sensor_data": {
$(generate_sensor_data)
    },
    "device_status": {
$(generate_device_status)
    },
    "metadata": {
        "sensor_type": "environmental",
        "unit_temperature": "Celsius",
        "unit_humidity": "%",
        "unit_pressure": "hPa",
        "unit_light_level": "lux",
        "firmware_version": "$((RANDOM % 3 + 1)).$((RANDOM % 10)).$((RANDOM % 10))"
    }
}
EOF
)

echo $json_data | jq -rc
}

for message_index in $(seq 0 $((NUM_MESSAGES_PER_DEVICE - 1))); do
    for device_index in $(seq 0 $((NUM_IDS - 1))); do
        publish_to_kafka "$device_index" "$message_index"
    done
done | kafka-console-producer.sh \
    --topic kafka-iot-topic \
    --bootstrap-server $1 \
    --producer.config $2

```

說明

- 這項指令碼會建立 JSON 訊息，其中包含模擬感應器讀數、裝置 ID、時間戳記、感應器資料 (溫度、濕度、壓力、光線)、位置資訊 (緯度、經度)、裝置狀態 (電池、訊號、連線類型) 和一些中繼資料。
- 這項工具會從一組不重複的裝置產生連續訊息流，每部裝置都會在指定時間間隔傳送資料，模擬真實的 IoT 裝置。在此，我們發布 10 部裝置的資料，每部裝置會以 10 秒的時間間隔產生 20 個讀數。
- 此外，這項工具也會使用 Kafka 生產者指令列工具，將所有產生的資料發布至 Kafka 主題。

5. 安裝指令碼使用的部分依附元件 - `bc` 套件用於數學計算，`jq` 套件用於處理 JSON。

```
sudo apt-get install bc jq
```

6. 將指令碼修改為可執行檔，然後執行指令碼。執行過程大約需要 2 分鐘。

```

chmod +x publish_iot_data.sh
./publish_iot_data.sh $BOOTSTRAP client.properties

```

執行下列指令即可列印所有事件，確認事件是否已順利發布。按下 `<control-c>` 即可退出。

```
kafka-console-consumer.sh \  
  --topic kafka-iot-topic \  
  --from-beginning \  
  --bootstrap-server $BOOTSTRAP \  
  --consumer.config client.properties
```

6. 設定 Dataproc 叢集

本節會在 Managed Kafka 叢集所在的虛擬私有雲子網路中，建立 Dataproc 叢集。這個叢集將用於執行作業，產生 DemoIoT Solutions 所需的即時統計資料和洞察資料。

注意：您可以結束與 GCE VM 的 SSH 工作階段，並在終端機中執行後續指令。確認終端機中已填入「總覽」一節的 `PROJECT_ID`、`REGION` 和 `SUBNET_PATH` 環境變數。

1. 建立 Dataproc 叢集。這裡的叢集名稱為 `dataproc-iot`。

```
gcloud dataproc clusters create dataproc-iot \
  --project=$PROJECT_ID \
  --region=$REGION \
  --image-version=2.2-debian12 \
  --enable-component-gateway \
  --subnet=$SUBNET_PATH \
  --master-machine-type=n1-standard-4 \
  --worker-machine-type=n1-standard-4 \
  --num-workers=2 \
  --properties=spark:spark.driver.memory=6G,spark:spark.driver.executor=4G
```

您應該會收到類似以下的回覆：

```
Waiting on operation [projects/<project-id>/regions/<region>/operations/<operation-id>].
```

```
Waiting for cluster creation operation...done.
```

```
Created [https://dataproc.googleapis.com/v1/projects/<project-id>/regions/<region>/clusters/dataproc-iot] Cluster placed in zone [<zone>].
```

建立叢集可能需要 10 到 15 分鐘。等待這項作業順利完成，然後說明叢集，確認叢集處於 `RUNNING` 狀態。

```
gcloud dataproc clusters describe dataproc-iot \
  --project=$PROJECT_ID \
  --region=$REGION
```

步驟 7 · 約 3 分鐘

7. 使用 Dataproc 處理 Kafka 訊息

在最後一節中，您將提交 Dataproc 工作，使用 [Spark Streaming](#) 處理發布的訊息。這項工作實際上會產生一些即時統計資料和洞察資訊，供 DemolIoT 解決方案使用。

注意：您可以在終端機中繼續執行這些指令。您需要存取 Google Cloud Storage bucket，這必須是在「總覽」的「必要條件」一節中建立。確認「總覽」部分的 `PROJECT_ID`、`REGION` 和 `BUCKET` 環境變數已填入。

1. 執行這項指令，在本機建立名為 `process_iot.py` 的串流 PySpark 工作檔案。

```
cat > process_iot.py <<EOF
```

```
#!/bin/python
```

```
import sys
import configparser
from google.cloud import storage
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, avg
from pyspark.sql.types import FloatType, IntegerType, StringType, StructField, StructType
from urllib.parse import urlparse
from pyspark.sql.utils import AnalysisException
```

```
JSON_SCHEMA = StructType([
    StructField("device_id", StringType()),
    StructField("timestamp", StringType()),
    StructField(
        "location",
        StructType([
            StructField("latitude", FloatType()),
            StructField("longitude", FloatType()),
        ]),
    ),
    StructField(
        "sensor_data",
        StructType([
            StructField("temperature", FloatType()),
            StructField("humidity", FloatType()),
            StructField("pressure", FloatType()),
            StructField("light_level", IntegerType()),
        ]),
    ),
    StructField(
        "device_status",
        StructType([
            StructField("battery_level", IntegerType()),
            StructField("signal_strength", IntegerType()),
            StructField("connection_type", StringType()),
        ]),
    ),
    StructField(
        "metadata",
        StructType([
            StructField("sensor_type", StringType()),
            StructField("unit_temperature", StringType()),
            StructField("unit_humidity", StringType()),
            StructField("unit_pressure", StringType()),
            StructField("unit_light_level", StringType()),
            StructField("firmware_version", StringType()),
        ]),
    ),
])
```



```

CLIENT_PROPERTY_KEYS = [
    "security.protocol",
    "saslm.echanism",
    "saslm.login.callback.handler.class",
    "saslm.jaas.config",
]

def get_client_properties(client_properties_path: str):
    # Parse client properties file
    parsed_path = urlparse(client_properties_path)
    if parsed_path.scheme != "gs":
        raise ValueError("Invalid GCS path for client properties. Must start with gs://.")

    bucket_name = parsed_path.netloc
    blob_name = parsed_path.path.lstrip("/")

    client = storage.Client()
    bucket = client.bucket(bucket_name)
    blob = bucket.blob(blob_name)
    file_content = "[DEFAULT]\n" + blob.download_as_text()

    config = configparser.ConfigParser()
    config.read_string(file_content)

    client_properties = dict()
    for key in CLIENT_PROPERTY_KEYS:
        client_properties[key] = config.get("DEFAULT", key)

    print(f"Client properties: {client_properties}")
    return client_properties

def process_initial_data(spark, bootstrap_server_address, client_properties,
queries_to_await, store_data_gcs_path):
    print("Starting initial data processing...")
    initial_rows = (
        spark.readStream.format("kafka")
        .option("kafka.bootstrap.servers", bootstrap_server_address)
        .option("startingOffsets", "earliest")
        .option("subscribe", "kafka-iot-topic")
        .option("kafka.security.protocol", client_properties["security.protocol"])
        .option("kafka.sasl.mechanism", client_properties["saslm.mechanism"])
        .option("kafka.sasl.login.callback.handler.class",
client_properties["saslm.login.callback.handler.class"])
        .option("kafka.sasl.jaas.config", client_properties["saslm.jaas.config"])
        .load()
    )

    initial_rows = (
        initial_rows.selectExpr("CAST(value AS STRING)")
        .select(from_json(col("value"), JSON_SCHEMA).alias("data"))
        .select("data.*")
    )

```

```

)

# Print first 20 rows
def print_top_rows(batch_df, batch_id):
    if batch_df.count() > 0:
        print(f"Processing initial batch for printing first 20 rows: batch {batch_id},
size: {batch_df.count()}")
        batch_df.limit(20).show(truncate=False)

initial_query_print = initial_rows.writeStream \
    .foreachBatch(print_top_rows) \
    .trigger(once=True) \
    .start()
queries_to_await.append(initial_query_print)

# Calculate and print average temperatures
def process_initial_avg_temp(batch_df, batch_id):
    if batch_df.count() > 0:
        print(f"Processing initial batch for avg temp: batch {batch_id}, size:
{batch_df.count()}")
        current_averages = (
            batch_df.select("device_id", "sensor_data.temperature")
            .groupBy("device_id")
            .agg(avg("temperature").alias("average_temperature"))
            .orderBy("device_id")
        )
        current_averages.show(truncate=False)

initial_query_avg_temp = initial_rows.writeStream \
    .foreachBatch(process_initial_avg_temp) \
    .trigger(once=True) \
    .start()
queries_to_await.append(initial_query_avg_temp)

# Write data to GCS
initial_data_gcs_writer = (
    initial_rows.writeStream.outputMode("append")
    .format("avro")
    .option("path", store_data_gcs_path+"/tables/iot_avro/")
    .option("checkpointLocation", store_data_gcs_path+"/chkpt/avro/")
    .trigger(once=True) \
    .start()
)
queries_to_await.append(initial_data_gcs_writer)

def main(bootstrap_server_address, client_properties_path, store_data_gcs_path):

    client_properties = get_client_properties(client_properties_path)

    # Create SparkSession
    spark = SparkSession.builder.appName("IotStreamingParser").getOrCreate()

```

```

queries_to_await = []

process_initial_data(spark, bootstrap_server_address, client_properties,
queries_to_await, store_data_gcs_path)

# Wait for all queries to terminate
for query in queries_to_await:
    try:
        query.awaitTermination()
    except Exception as e:
        print(f"Error awaiting query: {e}")
    finally:
        query.stop()

spark.stop()

if __name__ == "__main__":
    if len(sys.argv) < 4:
        print("Invalid number of arguments passed ", len(sys.argv))
        print(
            "Usage: ",
            sys.argv[0],
            " <bootstrap_server_address> <client_properties_path> <gcs_path_to_store_data>",
        )
        raise ValueError("Invalid number of arguments passed.")

    parsed_data_path = urlparse(sys.argv[3])
    if parsed_data_path.scheme != "gs":
        raise ValueError("Invalid GCS path for storing data. Must start with gs://.")

    main(sys.argv[1], sys.argv[2], sys.argv[3])

EOF

```

說明

- 這段程式碼會設定 PySpark 結構化串流工作，從指定的 Kafka 主題讀取資料。這項服務會使用提供的 Kafka 伺服器啟動位址，以及從 GCS 設定檔載入的 Kafka 設定，連線至 Kafka 代理程式並進行驗證。
- 首先，它會從 Kafka 讀取原始資料 (以位元組陣列串流的形式)，然後將這些位元組陣列轉換為字串，並使用 Spark 的 StructType 套用 json_schema，指定資料結構 (裝置 ID、時間戳記、位置、感應器資料等)。
- 這會將前 10 列資料顯示到控制台以供預覽、計算每個感應器的平均溫度，並以 avro 格式將所有資料寫入 GCS bucket。Avro 是以列為基礎的資料序列化系統，可將結構化資料有效率地儲存在緊湊的結構定義二進位格式中，並提供結構定義演進、語言中立性，以及適用於大規模資料處理的高壓縮率。

2. 建立 client.properties 檔案，並填入 Kafka 伺服器啟動位址的環境變數。

```
cat <<EOF> client.properties
security.protocol=SASL_SSL
sasl.mechanism=OAUTHBEARER
sasl.login.callback.handler.class=com.google.cloud.hosted.kafka.auth.GcpLoginCallbackHandler
sasl.jaas.config=org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginModule
required;
EOF
```

```
export BOOTSTRAP=$(gcloud managed-kafka clusters describe kafka-iot --project=$PROJECT_ID --
location=$REGION --format='value(bootstrapAddress)')
```

注意：您可以使用 `echo $BOOTSTRAP` 顯示啟動伺服器位址。格式應為 `bootstrap.kafka-iot.<region>.managedkafka.<project-id>:9092`。舊版 `gcloud` 可能沒有 `:9092` 後置字元。如果缺少 `:9092` 後置字串，請透過 `export BOOTSTRAP=$BOOTSTRAP:9092` 新增。

3. 將 `process_iot.py` 和 `client.properties` 檔案上傳至 Google Cloud Storage 值區，供 Dataproc 工作使用。

```
gsutil cp process_iot.py client.properties $BUCKET
```

4. 將 Dataproc 作業的一些依附元件 JAR 複製到 GCS bucket。這個目錄包含使用 Kafka 執行 Spark Streaming 工作所需的 JAR，以及從「[設定用戶端機器](#)」取得的 Managed Kafka 驗證程式庫及其依附元件。

```
gsutil -m cp -R gs://dataproc-codelabs/managed-kafka-dependencies/ $BUCKET
```

5. 將 Spark 工作提交至 Dataproc 叢集。

```
gcloud dataproc jobs submit pyspark \
  $BUCKET/process_iot.py \
  --project=$PROJECT_ID \
  --region=$REGION \
  --cluster=dataproc-iot \
  --properties=spark.jars=$BUCKET/managed-kafka-dependencies/* \
  -- $BOOTSTRAP $BUCKET/client.properties $BUCKET
```

系統會列印 Spark 驅動程式記錄。您也應該能在控制台中看到這些資料表記錄，以及儲存在 GCS 值區中的資料。

25/06/11 05:16:51 INFO AppInfoParser: Kafka version: 3.7.1
25/06/11 05:16:51 INFO AppInfoParser: Kafka commitId: e2494e6ffb89f828
25/06/11 05:16:51 INFO AppInfoParser: Kafka startTimeMs: 1749619011472

Processing initial batch for avg temp: batch 0, size: 200

Processing initial batch for printing first 20 rows: batch 0, size: 200

device_id	timestamp	location	sensor_data	device_status
metadata				
sensor-6	2025-03-01T00:03:00Z	{33.42, -119.66}	{20.9, 50.7, 1003.2, 525}	{70, -41, Wi-Fi}
		{environmental, Celsius, %, hPa, lux, 3.6.0}		
sensor-7	2025-03-01T00:03:00Z	{33.32, -119.4}	{20.1, 51.0, 1000.9, 611}	{47, -98, LoRaWAN}
		{environmental, Celsius, %, hPa, lux, 2.4.5}		
sensor-8	2025-03-01T00:03:00Z	{33.46, -119.74}	{20.8, 54.3, 1001.3, 734}	{70, -45, LoRaWAN}
		{environmental, Celsius, %, hPa, lux, 1.1.6}		
sensor-9	2025-03-01T00:03:00Z	{33.04, -119.38}	{20.9, 54.5, 1002.9, 956}	{91, -99, Cellular}
		{environmental, Celsius, %, hPa, lux, 2.7.4}		
sensor-0	2025-03-01T00:03:10Z	{33.22, -119.56}	{20.4, 53.0, 1000.2, 239}	{41, -95, LoRaWAN}
		{environmental, Celsius, %, hPa, lux, 2.1.3}		
sensor-1	2025-03-01T00:03:10Z	{33.62, -119.4}	{20.0, 53.8, 1000.2, 907}	{97, -84, Wi-Fi}
		{environmental, Celsius, %, hPa, lux, 3.3.1}		
sensor-2	2025-03-01T00:03:10Z	{33.56, -119.66}	{20.1, 51.6, 1004.8, 824}	{37, -36, Wi-Fi}
		{environmental, Celsius, %, hPa, lux, 2.5.1}		
sensor-3	2025-03-01T00:03:10Z	{33.32, -120.0}	{20.6, 52.5, 1004.1, 557}	{96, -70, Cellular}
		{environmental, Celsius, %, hPa, lux, 2.9.4}		
sensor-4	2025-03-01T00:03:10Z	{33.48, -119.36}	{20.2, 53.7, 1002.8, 818}	{22, -76, LoRaWAN}
		{environmental, Celsius, %, hPa, lux, 1.1.2}		
sensor-5	2025-03-01T00:03:10Z	{33.48, -119.42}	{20.7, 51.5, 1002.7, 310}	{92, -44, Wi-Fi}
		{environmental, Celsius, %, hPa, lux, 3.5.9}		
sensor-6	2025-03-01T00:03:10Z	{33.02, -119.8}	{20.4, 50.4, 1001.0, 190}	{36, -90, Cellular}
		{environmental, Celsius, %, hPa, lux, 1.4.5}		
sensor-7	2025-03-01T00:03:10Z	{33.16, -119.72}	{20.8, 54.2, 1004.9, 509}	{19, -38, Wi-Fi}
		{environmental, Celsius, %, hPa, lux, 2.9.8}		
sensor-8	2025-03-01T00:03:10Z	{33.52, -119.9}	{20.0, 54.5, 1003.8, 10}	{29, -31, Wi-Fi}
		{environmental, Celsius, %, hPa, lux, 1.0.5}		
sensor-9	2025-03-01T00:03:10Z	{33.64, -119.64}	{20.4, 52.4, 1003.7, 246}	{53, -64, Wi-Fi}
		{environmental, Celsius, %, hPa, lux, 1.7.3}		
sensor-8	2025-03-01T00:00:40Z	{33.46, -119.42}	{20.0, 52.8, 1003.1, 311}	{85, -67, Cellular}
		{environmental, Celsius, %, hPa, lux, 2.2.9}		
sensor-9	2025-03-01T00:00:40Z	{33.62, -119.98}	{20.0, 53.5, 1004.1, 502}	{22, -26, LoRaWAN}
		{environmental, Celsius, %, hPa, lux, 1.1.3}		
sensor-0	2025-03-01T00:00:50Z	{33.0, -119.38}	{20.1, 53.1, 1003.2, 500}	{49, -84, LoRaWAN}
		{environmental, Celsius, %, hPa, lux, 3.0.2}		
sensor-1	2025-03-01T00:00:50Z	{33.46, -119.48}	{20.1, 51.6, 1001.3, 982}	{52, -86, Wi-Fi}
		{environmental, Celsius, %, hPa, lux, 3.0.7}		
sensor-2	2025-03-01T00:00:50Z	{33.56, -119.74}	{20.3, 52.9, 1004.2, 367}	{29, -100, Cellular}
		{environmental, Celsius, %, hPa, lux, 3.2.6}		
sensor-3	2025-03-01T00:00:50Z	{33.54, -119.98}	{20.5, 51.2, 1004.2, 657}	{79, -50, LoRaWAN}
		{environmental, Celsius, %, hPa, lux, 1.0.0}		

--+-----+

25/06/11 05:17:10 INFO AppInfoParser: App info kafka.admin.client for adminclient-2
unregistered

25/06/11 05:17:10 INFO Metrics: Metrics scheduler closed

25/06/11 05:17:10 INFO Metrics: Closing reporter org.apache.kafka.common.metrics.JmxReporter

25/06/11 05:17:10 INFO Metrics: Metrics reporters closed

+-----+

|device_id|average_temperature|

+-----+

|sensor-0 |20.45999994277954 |

|sensor-1 |20.475 |

|sensor-2 |20.475 |

|sensor-3 |20.405000305175783 |

|sensor-4 |20.42000017166138 |

|sensor-5 |20.464999961853028 |

|sensor-6 |20.579999923706055 |

|sensor-7 |20.544999885559083 |

|sensor-8 |20.41999969482422 |

|sensor-9 |20.405000019073487 |

+-----+

8. 清除所用資源

完成 Codelab 後，請按照步驟清理資源。

1. 刪除代管 Kafka 叢集、發布者 GCE VM 和 Dataproc 叢集。

```
gcloud managed-kafka clusters delete kafka-iot --project=$PROJECT_ID --location=$REGION
gcloud compute instances delete publisher-instance --project=$PROJECT_ID --zone=$ZONE
gcloud dataproc clusters delete dataproc-iot --project=$PROJECT_ID --region=$REGION
```

2. 刪除虛擬私有雲子網路和網路。

```
gcloud compute networks subnets delete $SUBNET --region=$REGION
gcloud compute networks delete <network-name>
```

3. 如果不再需要資料，請刪除 GCS 值區。

```
gcloud storage rm --recursive $BUCKET
```

步驟 9 · 約 2 分鐘

9. 恭喜

恭喜！您已使用 Manage Kafka 和 Dataproc 成功建構 IoT 資料處理管道，協助 DemoIoT Solutions 即時掌握裝置發布的資料！

您已建立 Managed Kafka 叢集、將 IoT 事件發布至該叢集，並執行 Dataproc 工作，使用 Spark 串流即時處理這些事件。您現已瞭解，如要使用 Managed Kafka 和 Dataproc 建立資料管道，必須採取哪些重要步驟。

參考文件

- [Dataproc 簡介](#)
 - [Managed Service for Apache Kafka 總覽](#)
 - [使用 CLI 發布及接收訊息](#)
 - [提交 Dataproc 工作](#)
-